

Call Flow

Call hierarchies for implemented major flows. Paths are relative to `BankOne/src/main/java/com/bankone/` unless noted.

Login

```
AuthenticationController.login()
    ↓
AuthenticationService.login()
    ↓
AuthenticationManager.authenticate()
    ↓
CustomUserDetailsService.loadUserByUsername()
    ↓
UserRepository / UserRoleRepository
    ↓
JwtService.generateToken()
    ↓
(LoginAttemptService on failure)
```

Frontend: `AuthService.login()` → store token → navigate `/app/dashboard`.

Get current profile

```
AuthenticationController.me()
    ↓
AuthenticationService.getCurrentUserProfile()
    ↓
SecurityContext → UserRepository / roles
```

Create customer (optional first account)

```
CustomerController.createCustomer()
    ↓
CustomerServiceImpl.createCustomer()
    ↓
CustomerRepository.save()
    ↓
(if accountType present)
AccountService.openAccount() → see Open Account
```

Open account

```
AccountController.openAccount()
    ↓
AccountServiceImpl.openAccount()
    ↓
CustomerRepository.findById()
    ↓
AccountPolicyRepository.findByAccountTypeAndCurrencyCodeAndActiveTrue()
    ↓
validate opening deposit vs policy
    ↓
AccountRepository.getNextOrdinal()
    ↓
AccountNumberGenerator.generate()
    ↓
AccountRepository.save()
    ↓
(if openingDeposit > 0) TransactionService.record(CREDIT)
    ↓
NotificationEventPublisher.publish(ACCOUNT_OPENED, recipientEmail)
```

Frontend (Add Current):

```
CustomerDetail.addCurrentAccount()
    ↓
AccountPolicyService.getActivePolicy()
    ↓
OpeningDepositDialog (if required)
    ↓
AccountService.create / openAccount POST
```

Get active account policy

```
AccountPolicyController.getActivePolicy()
    ↓
AccountPolicyServiceImpl.getActivePolicy()
    ↓
AccountPolicyRepository.findByAccountTypeAndCurrencyCodeAndActiveTrue()
```

Deposit

```
AccountController.deposit()
    ↓
AccountServiceImpl.deposit()
    ↓
AccountRepository.findById()
    ↓
assert ACTIVE + amount > 0
    ↓
increment availableBalance, ledgerBalance, creditCount, timestamps
    ↓
TransactionService.record(..., CREDIT, ...)
    ↓
AccountRepository.save()
    ↓
NotificationEventPublisher.publish(DEPOSIT_COMPLETE, recipientEmail)
```

Frontend: `AccountList` / deposit dialog → `POST /accounts/{id}/deposit`.

Withdraw

```
AccountController.withdraw()
    ↓
AccountServiceImpl.withdraw()
    ↓
AccountRepository.findByIdForUpdate()
    ↓
assert ACTIVE + funds
    ↓
TransactionService.record(..., DEBIT, ...)
    ↓
AccountRepository.save()
    ↓
NotificationEventPublisher.publish(WITHDRAW_COMPLETE, recipientEmail)
```

Transfer

```
AccountController.transfer()
    ↓
AccountServiceImpl.transfer()
    ↓
lock both accounts (id order)
    ↓
DEBIT from + CREDIT to via TransactionService.record
    ↓
NotificationEventPublisher.publish(TRANSFER_SUCCESS, dest customer email)
```

Kafka notification → email

```
AccountServiceImpl (after commit of money movement)
    ↓
NotificationEventPublisher.publish(BankActionEvent)
    ↓
Kafka topic bankone.notifications
    ↓
NotificationEventConsumer
    ↓
NotificationEmailComposer
    ↓
NotificationMailService (SMTP / SendGrid)
```

Get account by id

```
AccountController.getAccountById()
    ↓
AccountServiceImpl.getAccountById()
    ↓
AccountRepository.findById()
    ↓
map → AccountResponse
```

Get transactions by account

```
AccountController.getTransactions()
    ↓
PageRequests.of(..., TX_SORT_FIELDS, "createdAt")
    ↓
TransactionServiceImpl.getByAccountId()
    ↓
AccountRepository.existsById() (else IllegalArgumentException)
    ↓
TransactionRepository.findByAccountAccountIdOrderByCreatedAtDesc()
    ↓
map → Page<TransactionResponse>
```

Search accounts

```
AccountController.searchAccounts()
    ↓
AccountServiceImpl.searchAccounts()
    ↓
AccountSpecification.matching()
    ↓
AccountRepository.findAll(spec, pageable)
```

Search customers

```
CustomerController.getCustomers()
    ↓
CustomerServiceImpl.searchCustomers()
    ↓
CustomerSpecification.containsText()
    ↓
CustomerRepository.findAll(spec, pageable)
```

Update account status

```
AccountController.updateAccountStatus()
    ↓
AccountServiceImpl.updateAccountStatus()
    ↓
AccountRepository.findById() / save()
```

Employee list / create (ADMIN)

```
UserController.*
    ↓
UserService.*
    ↓
UserRepository / UserRoleRepository / RoleRepository
```

Dashboard summary

```
DashboardController.getDashboard()
    ↓
DashboardServiceImpl
    ↓
CustomerRepository.count() / AccountRepository / UserRepository
    ↓
todayTransactionCount = 0 (stub)
```